# Experiment 3: Construction of an Inverted Index

| | | | |
|---|---|---|---|
| **Student Name:** | Student Name | **Roll / Student ID:** | EXP3-11 |
| **Class / Division:** | D17B | **Submission Date:** | 2026-09-22 |
| **Target Rolls:** | Roll Numbers 11 - 15 (Division C) | **Self-Evaluation:** | **0 / 10 (0%)** |

## 1. Aim & Learning Objectives

Aim: To construct and analyze an Inverted Index data structure for a collection of textual documents, explore linguistic preprocessing transformations (tokenization, stopword removal, stemming), and execute Boolean, phrase, and proximity queries with linear-time pointer-merge algorithms.

- Construct an Inverted Index data structure mapping vocabulary terms to documents, term frequencies (tf), and token positions.
- Trace the 4 core pipeline stages of index construction: Document Tokenization, Pair Extraction, Lexicographic Sorting, and Postings Inversion.
- Implement and visualize linear-time Boolean retrieval algorithms (AND, OR, NOT) using sorted posting list pointer intersections.
- Execute positional phrase queries and proximity searches by matching token offsets within identical document postings.
- Quantify the empirical effect of linguistic preprocessing (case folding, stopword removal, Porter stemming) on vocabulary size and index compression.
- Analyze collection-wide term frequency distributions and empirically validate Zipf's Law and Heaps' Law.

## 2. Theoretical Summary & Index Metrics

Active Corpus: Corpus 1: Information Retrieval & Search Engines
Constructed Inverted Index Vocabulary Size (|V|): 38 unique terms.
Total Postings Elements: 52 entries.
Algorithm Complexities: Boolean AND/OR Pointer Merge = O(L1 + L2); Phrase Adjacency = O(Pos1 + Pos2); Incidence Matrix Space = O(|V| * N); Inverted Index Space = O(|Postings|).

## 3. Recorded Experimental Session Trials

*No experimental trials logged in this session.*

## 4. Student Observations & Analytical Discussion

1. Stopword removal and Porter stemming significantly reduced vocabulary size |V|, validating Heaps' Law.
2. Conjunctive Boolean queries executed in sub-millisecond linear time O(L1 + L2) through sorted pointer intersection.
3. Positional postings allowed exact phrase verification with zero false positives by matching consecutive token offsets.
4. Empirical term frequency distribution conforms to the heavy-tailed power law modeled by Zipf's Law.

*Evaluator / Faculty Signature & Stamp*